

The Ultimate Web Development Handbook



Introduction to React

In this lecture, we will set the foundation for working with React using Vite. The goal is to understand what React is, why we use it, and how to quickly set up a React project with Vite.

What is React?

React is a popular JavaScript library for building user interfaces. Instead of manipulating the DOM directly, React allows you to build applications using reusable components. These components make your code more organized, modular, and easier to maintain.

Key benefits of React:

- Component-based structure
- Declarative code style
- Efficient updates with the virtual DOM
- Large ecosystem and community support

Why use Vite?

Traditionally, React apps were created using Create React App (CRA). While CRA works, it can feel slow and bloated. Vite is a modern alternative that offers:

- Extremely fast startup times
- Hot Module Replacement (HMR) for instant updates
- Simpler configuration
- Lightweight builds

Vite makes it much easier to get started with React development.

Setting Up Your Environment

Before we can use Vite, make sure you have the following installed:

- Node.js (LTS version recommended)
- npm (comes bundled with Node.js)

You can check if Node.js and npm are installed by running:

```
node -v  
npm -v
```

Creating a React Project with Vite

To create a new React project with Vite, run the following command in your terminal:

```
npm create vite@latest my-react-app
```

This will prompt you to choose a framework. Select **React** (or **React + JavaScript** if you see multiple options).

Once the project is created, navigate into the folder and install dependencies:

```
cd my-react-app  
npm install
```

Now, run the development server:

```
npm run dev
```

You should see a development URL, usually `http://localhost:5173`, where your React app will be running.

Recap

- React is a JavaScript library for building user interfaces using components.
- Vite is a modern build tool that makes React development faster and simpler.
- We created a new React app using Vite and ran it locally.

Project Setup and Folder Structure

In this lecture, we will explore the project created with Vite and understand the key files and folders. Knowing the structure will make it easier to navigate and organize your React application.

Vite Project Overview

After creating a project with Vite, you will see a structure similar to this:

```
my-react-app/  
├─ node_modules/  
├─ public/  
├─ src/  
│   ├─ App.css  
│   ├─ App.jsx  
│   ├─ index.css  
│   └─ main.jsx  
├─ .gitignore  
├─ index.html  
├─ package.json  
├─ vite.config.js  
└─ README.md
```

Let's break this down.

Important Files and Folders

- **index.html**

The entry point of the application. Unlike traditional setups, Vite uses this as the main HTML file. Notice the `<div id="root"></div>` where your React components will be rendered.

- **src/**

This folder contains your React code.

- `main.jsx` is the entry point for React. It mounts the root React component (`App.jsx`) into the `#root` div in `index.html` .
- `App.jsx` is your main React component. You can think of it as the starting point of your UI.
- `App.css` and `index.css` are for styling your components and global styles.

- **public/**

Any static files like images or icons can be placed here. Files in this folder are served directly without processing.

- **package.json**

Contains the project metadata, dependencies, and scripts. The important scripts are:

```
"scripts": {  
  "dev": "vite",  
  "build": "vite build",  
  "preview": "vite preview"  
}
```

- **vite.config.js** Vite's configuration file. For beginners, you won't need to change this much.

Running the Project

Use the following command to start the development server:

```
npm run dev
```

Open the provided local URL (usually `http://localhost:5173`) in your browser. You will see the default Vite + React starter page.

Recap

- Vite creates a simple, lightweight project structure.
- `index.html` contains the root element.
- `main.jsx` connects React with the DOM.
- `App.jsx` is the root component where your UI begins.

Understanding React Components

In this lecture, we will learn what React components are and how to create them. Components are the building blocks of any React application.

What is a Component?

A component is a reusable piece of code that represents part of the user interface. Components can be as small as a button or as large as an entire page.

React components make your code modular and easier to maintain.

Types of Components

React supports two types of components:

- **Function Components:** These are the modern and recommended way to write components.
- **Class Components:** An older way of writing components (not commonly used today). We will focus only on function components.

Creating a Simple Component

Open `App.jsx` and you will see something like this:

```
function App() {  
  return (  
    <div>  
      <h1>Hello Vite + React</h1>  
    </div>  
  )  
}
```

```
export default App
```

This is a function component named `App`. It returns JSX, which looks like HTML but is actually JavaScript.

JSX in Action

JSX allows us to write HTML-like code inside JavaScript. For example:

```
function Greeting() {  
  return <h2>Welcome to React</h2>  
}
```

Here, `Greeting` is a component that can be reused anywhere in your application.

Using Components

To use a component, simply include it inside another component like this:

```
function App() {  
  return (  
    <div>  
      <Greeting />  
    </div>  
  )  
}
```

Notice that we use `<Greeting />` as if it were an HTML tag. This is how components are composed to build larger applications.

Recap

- Components are reusable building blocks in React.
- Function components are the modern way of creating components.

- JSX looks like HTML but allows JavaScript expressions inside.
- You can include components inside other components.

Passing Data with Props

In this lecture, we will learn how to pass data to components using props. Props allow components to be dynamic and reusable.

What are Props?

Props (short for “properties”) are a way of passing data from a parent component to a child component. They make components more flexible because the same component can display different content depending on the props it receives.

Example: Card Component

Let’s create a simple `Card` component inside the `src` folder:

```
function Card(props) {  
  return (  
    <div style={{ border: "1px solid gray", padding: "10px", margin: "10px" }}>  
      <h3>{props.title}</h3>  
      <p>{props.description}</p>  
    </div>  
  )  
}  
  
export default Card
```

Here, `Card` accepts `title` and `description` as props and displays them.

Using the Card Component

Now open `App.jsx` and use the `Card` component:

```
import Card from "../Card"

function App() {
  return (
    <div>
      <Card title="React Basics" description="Learn the fundamentals of React." />
      <Card title="Props in React" description="Understand how to pass data into
components." />
    </div>
  )
}

export default App
```

Each `Card` is the same component, but it displays different content based on the props passed.

Props Are Read-Only

It's important to note that props are **read-only**. A component should never change its props. Instead, they should be used to display or work with the data received from the parent component.

Recap

- Props allow you to pass data into components.
- Props make components reusable and dynamic.
- Props are read-only and should not be modified inside the component.